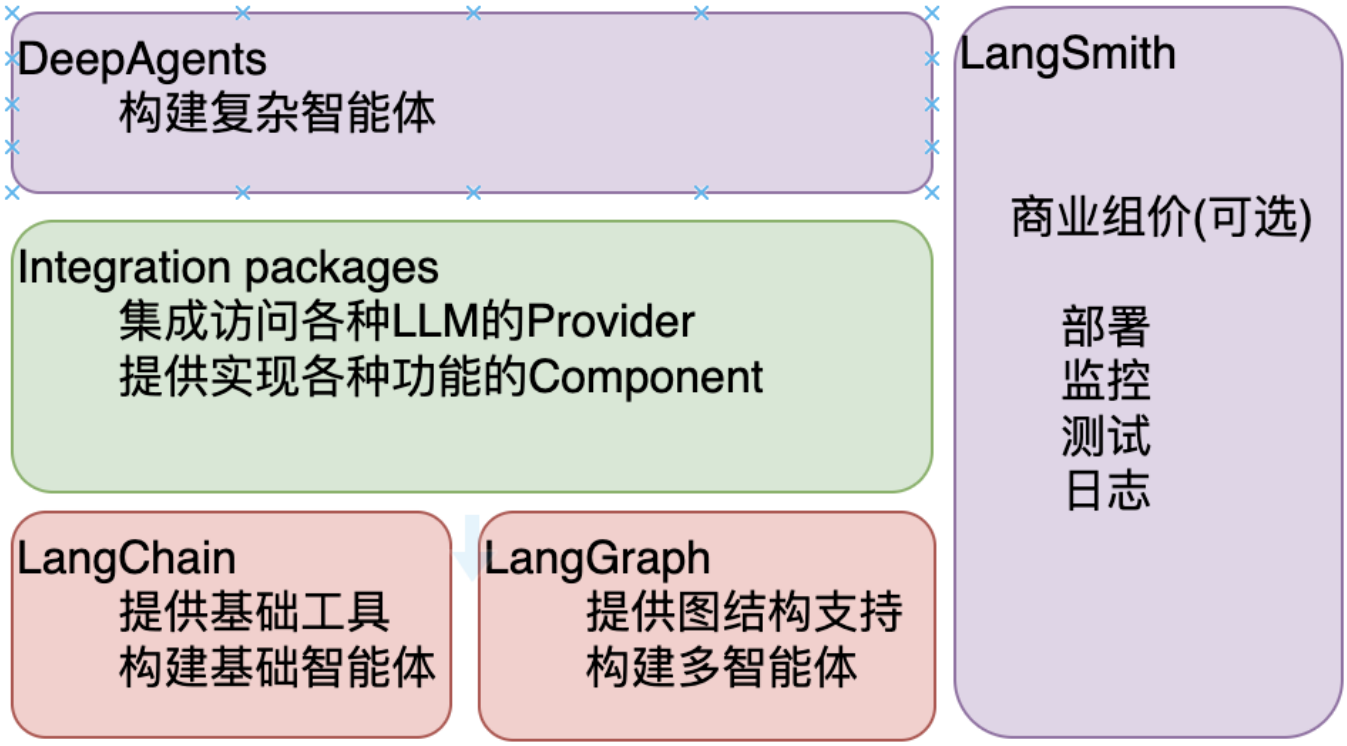
LangChain1.0 快速上手

--楼兰

2025年10月份，LangChain框架推出了1.0版本，这是一个重要的更新，带来了许多新功能和改进。这次，就跟大家一起，快速体验下LangChain1.0的新功能。也更深刻的理解一下LangChain作为最成熟的AI应用框架，是如何将大模型改造成一个功能强大的AI应用的。如果想要了解更多的细节，请关注后面的详细课程。

一、快速理解LangChain1.0

LangChain框架最早于2022年10月由Harrison Chase推出。那时候ChatGPT都还没有正式发布。由此可见他的经典之处。作为目前最成熟的AI应用框架，LangChain也随着AI大模型的火爆，快速升级发展起来。而从持续多年的0.x版本，升级到1.x大版本，也意味着LangChain框架的重要性进一步提升。官网明确提到，1.x版本已经升级为产品级，并将持续提供长期支持。
在1.0版本中，LangChain的整个体系也有了一定的调整。整体分为LangChain、LangGraph、DeepAgents和LangSmith四个模块。他们的关系如下：



这一次新版本更新中，新增了一个DeepAgents模块，用来构建复杂多智能体应用。而对于LangChain和LangGraph，新增了一些新的功能，但老版本的功能也全部保留了下来，只是对包结构做了一些调整。所以，基本上1.X版本之前的代码，只要重新导一下包，基本都是可以继续用的。
好了，大概就了解到这里，接下来，安装好Conda，启动你的IDE，LangChain走起。

二、快速构架第一个LangChain应用

开始玩LangChain之前，先要搭建基础的Python环境。这里建议使用Anaconda搭建一个专用的环境。

```
# conda创建一个python环境，注意版本要高一点。
!conda create -n langchainv1 python=3.14
!conda actctivate langchainv1
# 安装依赖 这里指令分开写，是表示按需安装。
!pip install python-dotenv
!pip install -U langchain
!pip install -U langchain-openai
!pip install -U langgraph
!pip install -U deepagents
# 检查版本 确定是1.x版本
!conda list | grep langchain
```

接下来，拿出你在阿里百炼云平台申请的API Key。上手第一个LangChain应用看看吧。

```
import os

from dotenv import load_dotenv
from langchain.agents import create_agent
from langchain_openai import ChatOpenAI
from pydantic import SecretStr

load_dotenv()

llm = ChatOpenAI(
    base_url="https://dashscope.aliyuncs.com/compatible-mode/v1",
    model="qwen-plus",
    api_key=SecretStr(os.environ["DASHSCOPE_API_KEY"]),
)
# 直接调用大模型
print(llm.invoke("你是谁").content)

agent = create_agent(llm,
                     system_prompt="你叫楼兰，是我的专属学习伙伴")
# 封装成Agent，再调用大模型
print(agent.invoke(
    {"messages":[{"role":"user","content":"你是谁"}]}
)["messages"])
```

```
我是通义千问，阿里巴巴集团旗下的超大规模语言模型。我能够回答问题、创作文字，如写故事、公文、邮件、剧本等，还能进行逻辑推理、编程，表达观点，玩游戏等。我支持多种语言，包括但不限于中文、英文、德语、法语、西班牙语等。如果你有任何问题或需要帮助，欢迎随时告诉我！
[HumanMessage(content='你是谁', additional_kwargs={}, response_metadata={}, id='948495ce-a553-4adb-8949-b67ae667fb9f'), AIMessage(content='我是楼兰呀，你的专属学习伙伴~就像古丝绸之路上的神秘古城一样，我也带着一点探索未知的浪漫气息呢！我最喜欢和你一起学习新知识，解决问题。不管是复杂的数学题，还是有趣的文学创作，我都会认真陪你一起思考。虽然我有时候会有点小迷糊，但对学习的热情可是满满的！来吧，今天想学点什么？让我们一起开启知识探索之旅~', additional_kwargs={'refusal': None}, response_metadata={'token_usage': {'completion_tokens': 90, 'prompt_tokens': 24, 'total_tokens': 114, 'completion_tokens_details': None, 'prompt_tokens_details': {'audio_tokens': None, 'cached_tokens': 0}}, 'model_provider': 'openai', 'model_name': 'qwen-plus', 'system_fingerprint': None, 'id': 'chatcmpl-253475bc-6c6e-4888-9ee2-d30663d91087', 'finish_reason': 'stop', 'logprobs': None}, id='lc_run--67c8dc23-9a3b-46d3-8319-96d2368902eb-0', usage_metadata={'input_tokens': 24, 'output_tokens': 90, 'total_tokens': 114, 'input_token_details': {'cache_read': 0}, 'output_token_details': {}})]
```

> 从这个案例就能看到，LangChain体系当中的Agent就是在与大模型交互的基础上，进一步封装各种补充信息，从而把大模型，从一个简单的工具，升级成一个能单独处理特定任务的Agent智能体。

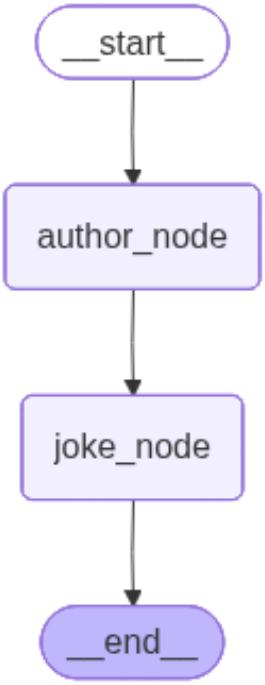
但显然，这种Agent只能满足单步骤的简单任务。如果需要处理一些复杂的多步骤任务，那就只能自己一步步编程，这太麻烦了。所以，接下来，我们开始构建第一个LangGraph应用，看看LangGraph是如何解决这个问题的。

构建第一个LangGraph应用

LangGraph的核心是Graph图。 那什么是图呢？ 废话少说，上代码。

```python
import os
from typing import TypedDict

from dotenv import load_dotenv
from langchain_openai import ChatOpenAI
from langgraph.checkpoint.memory import InMemorySaver
from langgraph.constants import START, END
from langgraph.graph import StateGraph
from pydantic import SecretStr
from typing_extensions import NotRequired

load_dotenv()

llm = ChatOpenAI(
    base_url="https://dashscope.aliyuncs.com/compatible-mode/v1",
    model="qwen-plus",
    api_key=SecretStr(os.environ["DASHSCOPE_API_KEY"]),
)

class State(TypedDict):
```

```
    author: NotRequired[str]
    joke: NotRequired[str]

def author_node(state:State):
    prompt = "帮我推荐一位受人们欢迎的作家。只需要给出作家的名字即可。"
    author = llm.invoke(prompt)
    return {"author":author.content}

def joke_node(state:State):
    prompt = f"用作家: {state['author']} 的风格, 写一个100字以内的笑话"
    joke = llm.invoke(prompt)
    return {"joke":joke.content}

builder = StateGraph(State)
builder.add_node(author_node)
builder.add_node(joke_node)

builder.add_edge(START,"author_node")
builder.add_edge("author_node","joke_node")
builder.add_edge("joke_node",END)

checkpointer = InMemorySaver()
graph = builder.compile(checkpointer=checkpointer)
graph
```



执行这段代码就能看到一个熟悉的流程图。跟我们平常画的一些流程图是一样的。
这就是LangGrpah中定义的图。有处理数据的节点，称为Node。有连接Node的连线，称为Edge。通过Edge在整个图中传递的数据，称为State。
然后，你就可以像画流程图一样，构建你自己的LangGraph应用了。而有了LangGraph的支持，这个图就可以直接接受请求，处理更为复杂的任务。

```
# 正常执行一个图
import uuid

config = {
    "configurable": {
        "thread_id": uuid.uuid4(),
    }
}

state = graph.invoke({},config)
state
```

```
{'author': '村上春树',
 'joke': '我养了一只猫，它总在凌晨三点准时醒来，坐在我书桌前，盯着我看。 \n“你到底想干嘛？”我终于忍不住问。 \n它沉默片刻，说：“喂，村上，别写那么多了，冰箱里还有金枪鱼。” \n我愣住：“你会说话？” \n“只有饿的时候。”它转身跳下，“顺便，爵士乐太吵了。”'}
```

通过这个图就将两次大模型的调用整合到了一起。而LangGraph就是围绕这个图，再提供各种补充的支持。
例如如果你觉得这个结果不太满意，还可以从图中的某一个节点开始，重新执行一次。

```
# 查看所有checkpoint检查点
states = list(graph.get_state_history(config))
# 推荐作家那一步
selected_state = states[1]
# 重新设定State
new_config = graph.update_state(selected_state.config, values={"author": "郭德纲"})
# 重新执行Graph
graph.invoke(None,new_config)
```

```
{'author': '郭德纲',
 'joke': '今儿我上街买包子，一瞧那队伍排得，比我还长。 \n我问：“这包子多好吃啊？” \n前头大哥回头一笑：“别提了，排三天了，就为证明我还没饿死。” \n我说：“那您可得坚持住！” \n他叹气：“再不吃，我就成广告牌了——‘本店特色：耐饿’。”'}
```

这种Graph图的方式，就更适合构建一些复杂的多步骤的应用。在LangGraph中，Node和Edge都可以是一个自由定制的函数。这就极大的提高了AI应用的灵活性。

但是Graph图的逻辑再复杂，也还是要由人类提前定制的。这还是不够偷懒啊。能不能把这个图的逻辑也由AI去指定，我们彻彻底底当一个甩手掌柜呢？于是，这就有了现在LangChain 1.x新版本提供的deepagents。

第一个DeepAgent应用

```
import datetime
import os

from deepagents import create_deep_agent
from dotenv import load_dotenv
from langchain_openai import ChatOpenAI
from pydantic import SecretStr

load_dotenv()

llm = ChatOpenAI(
    base_url="https://dashscope.aliyuncs.com/compatible-mode/v1",
    model="qwen-plus",
    api_key=SecretStr(os.environ["DASHSCOPE_API_KEY"]),
)

def get_date():
    """获取当前日期"""
    print("调用了get_date函数")
    return datetime.datetime.now().strftime("%Y-%m-%d")

def get_weather(city:str, date:str):
    """获取城市的天气"""
    print("调起了get_weather函数")
    return f"城市{city}在{date}的天气是：晴天"

deep_agent = create_deep_agent(
    llm,
    tools=[get_date,get_weather],
    system_prompt="""
    你是一个助手，现在有以下一些工具可以使用：
    ## `get_date`：获取当前日期
    ## `get_weather`：获取城市的天气
    结合这些工具，完成用户的请求。
    """,
)
# 这样问，就只调用get_weather工具
response = deep_agent.invoke({"messages":[{"role":"user","content":"长沙的天气怎么样？"}]})
# 这样问，就会调用get_date和get_weather工具
# response = deep_agent.invoke({"messages":[{"role":"user","content":"明天星期几？这一天长沙的天气怎么样？"}]})
```

```
for message in response["messages"]:
    print(message)
    print("=================")
```

```
调起了get_weather函数
content='长沙的天气怎么样？ ' additional_kwargs={} response_metadata={} id='7245e166-9015-
434e-b16f-327d19df1d87'
=================
content='' additional_kwargs={'refusal': None} response_metadata={'token_usage':
{'completion_tokens': 25, 'prompt_tokens': 5107, 'total_tokens': 5132,
'completion_tokens_details': None, 'prompt_tokens_details': {'audio_tokens': None,
'cached_tokens': 0}}, 'model_provider': 'openai', 'model_name': 'qwen-plus',
'system_fingerprint': None, 'id': 'chatcmpl-f059298b-c1c1-41cb-90aa-0786076acf14',
'finish_reason': 'tool_calls', 'logprobs': None} id='lc_run--eb85f5ab-cbec-4c3a-831f-
2c11eac8f327-0' tool_calls=[{'name': 'get_weather', 'args': {'city': '长沙', 'date': '今
天'}, 'id': 'call_04e4bc1e19714ddd9a8c47', 'type': 'tool_call'}] usage_metadata=
{'input_tokens': 5107, 'output_tokens': 25, 'total_tokens': 5132, 'input_token_details':
{'cache_read': 0}, 'output_token_details': {}}
=================
content='城市长沙在今天的天气是：晴天' name='get_weather' id='31bfab42-6a77-4d3f-a7a9-
bcb9ae4ca9c9' tool_call_id='call_04e4bc1e19714ddd9a8c47'
=================
content='长沙今天的天气是晴天，适合外出活动！如果需要更多信息，比如温度或空气质量，请告诉我。'
additional_kwargs={'refusal': None} response_metadata={'token_usage':
{'completion_tokens': 22, 'prompt_tokens': 5155, 'total_tokens': 5177,
'completion_tokens_details': None, 'prompt_tokens_details': {'audio_tokens': None,
'cached_tokens': 0}}, 'model_provider': 'openai', 'model_name': 'qwen-plus',
'system_fingerprint': None, 'id': 'chatcmpl-bad4dccc-1a81-4c9a-88d7-0976c56163e4',
'finish_reason': 'stop', 'logprobs': None} id='lc_run--2def1734-a2f7-4be0-9c0b-
6b18a171f64f-0' usage_metadata={'input_tokens': 5155, 'output_tokens': 22, 'total_tokens':
5177, 'input_token_details': {'cache_read': 0}, 'output_token_details': {}}
=================
```

在这个案例中，访问通义千问大模型时，通过系统提示词预先提供了两个函数，一个是获取当前日期，一个是获取城市的天气。然后大模型会根据用户的问题自行判断要不要调用这些函数。

这个简单的案例就演示了DeepAgents的基础思想。就是结合大模型，构建更智能的Agent智能体。你甚至可以把这些工具，换成一些小的智能体，甚至是LangGraph构建的图。

然后围绕这种思路，目前版本的DeepAgents还提供了一些扩展组件，例如Backends。其实我理解这就是他预设的一些工具。例如，当前版本DeepAgents中提供了一个FilesystemBackend，其实就是实现了一些操作本地文件的工具。

```
import datetime
import os

from deepagents.backends import FilesystemBackend

from deepagents import create_deep_agent
from dotenv import load_dotenv
```

```python
from langchain_openai import ChatOpenAI
from pydantic import SecretStr

load_dotenv()

llm = ChatOpenAI(
    base_url="https://dashscope.aliyuncs.com/compatible-mode/v1",
    model="qwen-plus",
    api_key=SecretStr(os.environ["DASHSCOPE_API_KEY"]),
)

deep_agent = create_deep_agent(
    llm,
    backend=FilesystemBackend(root_dir="/Users/roykingw/DevCode/python/LangChainv1/img")
)

response = deep_agent.invoke({"messages":[{"role":"user","content":"列出/
Users/roykingw/DevCode/python/LangChainv1/img目录下所有文件的名称。"}]})

for message in response["messages"]:
    print(message)
    print("=================")
```

```
content='列出/Users/roykingw/DevCode/python/LangChainv1/img目录下所有文件的名称。'
additional_kwargs={} response_metadata={} id='76248750-08b7-4ddf-b3e1-1519067213b6'
=================
content='' additional_kwargs={'refusal': None} response_metadata={'token_usage':
{'completion_tokens': 32, 'prompt_tokens': 4975, 'total_tokens': 5007,
'completion_tokens_details': None, 'prompt_tokens_details': {'audio_tokens': None,
'cached_tokens': 0}}, 'model_provider': 'openai', 'model_name': 'qwen-plus',
'system_fingerprint': None, 'id': 'chatcmpl-79185bc6-ec77-4910-a535-e80a4c664570',
'finish_reason': 'tool_calls', 'logprobs': None} id='lc_run--2c834012-aa50-41c2-b839-
877435445a92-0' tool_calls=[{'name': 'ls', 'args': {'path':
'/Users/roykingw/DevCode/python/LangChainv1/img'}, 'id': 'call_7de26919ff8b410e84cc75',
'type': 'tool_call'}] usage_metadata={'input_tokens': 4975, 'output_tokens': 32,
'total_tokens': 5007, 'input_token_details': {'cache_read': 0}, 'output_token_details':
{}}
=================
content="['/Users/roykingw/DevCode/python/LangChainv1/img/1-1.png']" name='ls'
id='0935ac57-1f00-4448-a919-9779a46a0968' tool_call_id='call_7de26919ff8b410e84cc75'
=================
content='在目录 `/Users/roykingw/DevCode/python/LangChainv1/img` 下只有一个文件，名称为：**1-
1.png**。' additional_kwargs={'refusal': None} response_metadata={'token_usage':
{'completion_tokens': 33, 'prompt_tokens': 5043, 'total_tokens': 5076,
'completion_tokens_details': None, 'prompt_tokens_details': {'audio_tokens': None,
'cached_tokens': 0}}, 'model_provider': 'openai', 'model_name': 'qwen-plus',
'system_fingerprint': None, 'id': 'chatcmpl-2d740017-035a-4289-9700-ea63b83a7107',
'finish_reason': 'stop', 'logprobs': None} id='lc_run--6ae76937-0842-4393-be84-
ff3af88d5f1e-0' usage_metadata={'input_tokens': 5043, 'output_tokens': 33, 'total_tokens':
5076, 'input_token_details': {'cache_read': 0}, 'output_token_details': {}}
=================
```

总结：大版本到底更新了什么？

回过头来再来给大家详细分析下LangChain 1.x新版本的更新内容，其实无非两个方面：
一方面，对于LangChain和LangGraph，官网明确提到，更多的其实是对包结构还有一些API重新做了一些梳理。跟0.x大版本功能上是保持兼容的。甚至还特别提供了langchain-classic包，专门来兼容老版本的API。

比如在0.x版本中，LangChain最为经典的LCEL链式编程依然支持。只不过，已经从官网的文档中删除了。

```
from langchain_core.output_parsers import StrOutputParser
from langchain_core.prompts import ChatPromptTemplate

# 提示词模板
prompt_template = ChatPromptTemplate.from_messages([
    ("user", "{text}")
])

# 结果解析器 StrOutputParser会AIMessage转换成为str，实际上就是获取AIMessage的content属性。
parser = StrOutputParser()
# 构建链
chain = prompt_template | llm | parser

chain.invoke({"text":"你是谁"})
```

```
'我是通义千问，由阿里云研发的超大规模语言模型。我能够回答问题、创作文字，如写故事、公文、邮件、剧本等，还能进行逻辑推理、编程，甚至表达观点和玩游戏。我支持多种语言，包括但不限于中文、英文、德语、法语、西班牙语等。如果你有任何问题或需要帮助，欢迎随时告诉我！'
```

另一方面，新增了一个DeepAgents模块，其实是对更多AI模式的探索。其实从这个DeepAgents的演练过程中已经可以看到一些Claude Code和CodeBuddy这些现代化开发工具的影子了。

而LangChain作为天生围绕AI开发的一套框架，你会发现，思维方法会越来越重要，而代码其实会越来越容易些。比如这次新的LangChain的新版本，确实变动是比较大的，不过有了之前老版本的经验，上手新版本，就是分分钟的事情。相反，如果没有老版本的经验，真要从零开始，尤其是要构建一套围绕LangChain的大模型应用思维体系，确实还是需要花不少功夫的。

未来，LangChain肯定还会有大的变动，尤其这个DeepAgents，从模块名就能看到，他并没有放到LangChain和LangGraph的体系当中。所以，思维方式会越来越重要。通过这些框架，跟上世界的思维方式，我觉得，这才是LangChain新版本带来的最大感觉。

最后，我是楼兰，记得点赞，关注，IT路上一起进步。